

Cloud computing Project

Application: **CloudDash Integrity**

Cloud Computing - Dashcam Encoder / Transmitter and Decoder

Student: **NHEMBO Ashriel**

Complete Project Documentation

Hybrid Dashcam Evidence Acquisition, Transmission and Integrity Verification

Introduction

This document is the complete technical documentation for the CloudDash Integrity project. It covers the Encoder/Transmitter, Decoder/Insurer, development environment, installation, Supabase configuration, architecture, implementation choices, integrity protocol, offline behavior, retention, deployment, automated tests, runtime evidence, demonstration plan, limitations and final deliverables.

Table of Contents

1. Project Overview
2. Requirements and Deliverables
3. Technology Stack and Repository Structure
4. System Architecture
5. Development Environment and Installation
6. Supabase Backend Configuration
7. Encoder / Transmitter Implementation
8. Offline Transmission and Retention
9. Incident Evidence Workflow
10. Decoder / Insurer Implementation
11. Integrity Verification Mechanism
12. Security and Access Control
13. Deployment Architecture
14. Testing and Validation
15. Runtime Demonstration Evidence
16. Demonstration Video Procedure
17. Requirement Traceability
18. Known Limitations and Production Hardening
19. Conclusion
- Appendix A. Repository File Map
- Appendix B. Reproducibility Commands

1. Project Overview

CloudDash Integrity is a hybrid dashcam evidence system designed for a driver smartphone and an insurer-side verification workflow. The system deliberately separates continuous video storage from integrity metadata. During normal driving, video remains local on the driver's device while cryptographic fingerprints are transmitted to the cloud. When the driver locks an incident, a bounded incident window is preserved, and the corresponding video is additionally uploaded to private cloud storage.

The insurer-side Decoder verifies that a supplied or retrieved video is identical to the originally fingerprinted evidence. Verification combines exact-byte SHA-256 hashing, ECDSA P-256 signatures, capture-session identity, ordered hash-chain continuity and the trusted Supabase record.

Core design principle

Normal driving video is local-first. The cloud receives trusted signed fingerprints and session metadata. Only protected incident video is stored in the private evidence bucket.

2. Requirements and Deliverables

Component	Required deliverable	Project response
Encoder / Transmitter	Complete application and source code	React/Vite application, local storage, cryptographic modules, Supabase repositories and migrations
Encoder / Transmitter	Installation and environment report	Documented in README, SUPABASE.md, DEPLOYMENT.md and this report
Encoder / Transmitter	Implementation and technical choices	Camera/simulation capture, MediaRecorder segments, SHA-256, ECDSA signatures, hash chain, IndexedDB outbox, retention
Encoder / Transmitter	Video demonstration	Procedure documented; final recording must be submitted
Decoder / Insurer	Complete application and source code	Decoder, cloud reference stream, local file verification and private incident retrieval are implemented
Decoder / Insurer	Installation and environment report	Same reproducible application/backend environment
Decoder / Insurer	Integrity mechanism explanation	Exact-byte hashing + signature verification + session hash-chain verification + trusted cloud record
Decoder / Insurer	Video demonstration	Procedure documented; final recording must be submitted

3. Technology Stack and Repository Structure

Layer	Technology / implementation
Frontend	React 18, React Router, Vite 6
Video capture	Browser MediaRecorder and Media APIs
Cryptography	Web Crypto API: SHA-256 and ECDSA P-256 with SHA-256
Local persistence	IndexedDB for video blobs, device key pair and offline fingerprint outbox
Cloud backend	Supabase Auth, PostgreSQL, Realtime and Storage
Authorization	Supabase Row Level Security with workspace/device access rules
Deployment	Vercel SPA deployment; optional Node/Express Docker

Layer	Technology / implementation
	deployment
Validation	Node test runner under tests/ and GitHub Actions CI

Key repository locations:

- src/App.jsx - application UI and end-to-end Encoder/Decoder workflows
- src/security/chain.js - evidence canonicalization, SHA-256 and chain verification
- src/security/deviceKeys.js - device signing key generation and ECDSA signing/verification
- src/security/verification.js - file hash comparison and trusted fingerprint verification
- src/storage/localEvidenceStore.js - local video persistence and retention
- src/storage/offlineQueue.js - persistent retry queue
- src/fingerprintRepository.js - fingerprint, session and incident cloud operations
- supabase/migrations/ - database schema, RLS and v2 capture-session hardening
- docs/ - architecture, deployment, Supabase setup and demo documentation
- tests/ - integrity, repository and evidence file tests

4. System Architecture

Figure 1 presents the complete hybrid evidence architecture. It separates the driver smartphone, on-device cryptographic processing and local storage, Supabase cloud services, and the insurer-side Decoder.

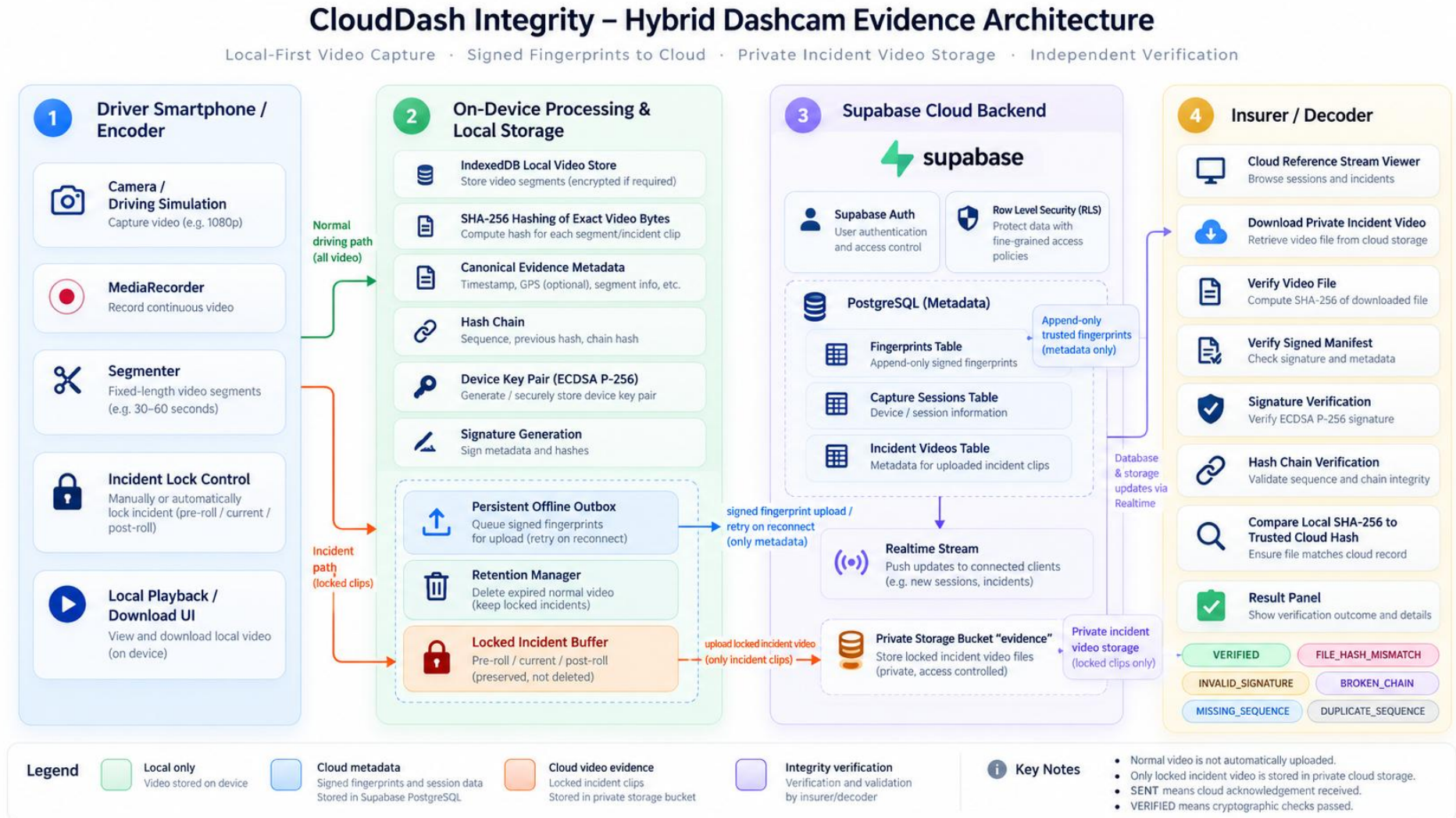


Figure 1. CloudDash Integrity hybrid dashcam evidence architecture. Normal video remains local; signed fingerprints persisted in Supabase; only locked incident video is uploaded to private evidence storage; the Decoder independently validates the file hash, device signature and capture-session chain.

5. Development Environment and Installation

5.1 Prerequisites

- Node.js 22 or a compatible modern Node release and npm.
- Git for source retrieval and version control.
- A modern Chromium-based browser for camera/media APIs, Web Crypto and IndexedDB.
- Docker Desktop and Supabase CLI if reproducing the full local Supabase environment.
- A Supabase hosted project for cloud deployment.
- A Vercel account for the documented frontend deployment path.

5.2 Clone and install

```
git clone https://github.com/asrieldev/clouddash-integrity.git
cd clouddash-integrity
npm install
```

5.3 Environment variables

Create a .env file in the repository root:

```
VITE_SUPABASE_URL=<your Supabase project URL>
VITE_SUPABASE_ANON_KEY=<your public browser/anon key>
```

These are browser configuration values used by the Supabase client. Service-role or other privileged server secrets must not be embedded in the frontend.

5.4 Start the application

```
npm run dev
```

For the optional combined local frontend/server workflow:

```
npm start
```

6. Supabase Backend Configuration

6.1 Local Supabase

1. Install the Supabase CLI and Docker Desktop.
2. Run `supabase start` from the repository root.
3. Run `supabase db reset` to apply the migration history.
4. Copy the local project URL and anon key into .env.
5. Start the frontend with `npm run dev`.

6.2 Hosted Supabase

6. Create the hosted Supabase project.
7. Link the repository using `supabase link --project-ref <project-ref>`.
8. Apply the schema using `supabase db push`.
9. Configure `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` in the deployment environment.
10. Configure the deployed application origin in Supabase Authentication Site URL and Redirect URLs.

6.3 Core cloud tables and services

Object	Role in CloudDash
devices	Stores device identity, ownership, public signing key and device metadata
capture_sessions	Separates independent recording sessions and their lifetime
fingerprints	Append-only trusted evidence references: hash, sequence, signature, chain fields
incidents	Represents driver-locked events
incident_videos	Links a private Storage object to its trusted fingerprint/session/segment
Storage bucket: evidence	Private object storage for locked incident video only
Supabase Realtime	Delivers cloud reference updates to connected interfaces

7. Encoder / Transmitter Implementation

7.1 Capture and segmentation

Each Start Dashcam action creates a new capture session. MediaRecorder captures from the selected camera or driving simulation source and divides the recording into fixed-length evidence segments. The UI supports configurable segment length, incident pre-roll/post-roll and local retention.

7.2 Local persistence

Before cloud transmission, original video blobs and evidence metadata are persisted in IndexedDB. The local store supports playback, download, per-device listing, size accounting and deletion. Normal video can therefore remain available through page refreshes while the cloud only stores its trusted reference.

7.3 Exact-byte hashing

SHA-256 is computed using Web Crypto over the exact video segment bytes. Any changed byte produces a different digest. This means re-encoding, blur, crop, overlays, frame edits, compression changes, audio edits, resolution changes or file replacement will cause an exact-byte mismatch.

7.4 Canonical metadata and hash chain

Evidence version 2 binds workspaceId, deviceId, sessionId, segmentId, sequence, normalized capturedAt, bytes, SHA-256 and previousHash. The chainHash is the SHA-256 digest of this canonical representation. The first v2 segment must start at sequence 0 with previousHash = null.

7.5 ECDSA device signature

A P-256 ECDSA key pair is generated with Web Crypto. The private key remains in the device-side IndexedDB key store. The canonical fingerprint payload, including the chain hash, is signed using ECDSA with SHA-256. The public key is registered with the device for Decoder verification.

8. Offline Transmission and Retention

8.1 Persistent outbox

The offline queue is stored in IndexedDB. Fingerprints move through queue states such as QUEUED, SENDING and FAILED; successful transmission removes the queue entry and the UI reflects the segment as SENT. A browser refresh or temporary connection loss does not discard pending evidence.

8.2 Idempotency and conflict handling

Each modern segment uses an immutable segment UUID. Capture-session/sequence uniqueness prevents accidental cross-session collision. If an insert encounters an existing identical record, repository code can treat it as idempotent; conflicting evidence identity fails rather than overwriting the trusted record.

8.3 Retention

The retention manager deletes expired normal video while preserving locked evidence. Manual deletion is guarded so a normal recording is only deletable once its fingerprint is successfully acknowledged and the recording is not locked.

Difference between SENT and VERIFIED

SENT means that Supabase acknowledged the fingerprint insert. VERIFIED is a Decoder result and requires the cryptographic verification conditions to pass.

9. Incident Evidence Workflow

Lock Incident Clip protects a bounded evidence window around an event. The selected pre-roll/current/post-roll local segments are marked as locked and therefore excluded from rolling retention.

After the corresponding fingerprint has been acknowledged, the locked video is uploaded to the private evidence bucket with upsert disabled. The incident_videos row links storage_path, fingerprint_id, session_id, segment_id, sequence, hash and signature. Existing-object conflicts are validated by re-hashing the existing bytes and fail closed if they differ from the trusted fingerprint.

10. Decoder / Insurer Implementation

The Decoder is the insurer-side verification interface. It can verify a local .webm or .mp4 file, validate a signed JSON evidence manifest and retain support for legacy raw .txt hashes in review mode. Local video hashing occurs inside the browser; a locally supplied verification file is not uploaded merely to check its hash.

10.1 Cloud reference stream

Trusted fingerprints are loaded from Supabase and updated through Realtime. Modern records are grouped by session identity, so verification is scoped to the exact recording session rather than all sequences produced by a device.

10.2 Private incident retrieval

Authorized users can retrieve incident videos from the private evidence Storage bucket, play/download it and verify it against the linked trusted fingerprint.

11. Integrity Verification Mechanism

11.1 Verification sequence

11. Compute SHA-256 of the supplied video bytes.
12. Resolve the trusted fingerprint record in Supabase.
13. Compare the local digest with the trusted cloud SHA-256 value.
14. Load the public key associated with the trusted record.
15. Verify the ECDSA P-256 signature over canonical fingerprint metadata.
16. Load all applicable session records from sequence 0 through the target sequence.
17. Recompute and validate the ordered hash chain, sequence continuity and previous-hash links.
18. Return VERIFIED only if the applicable cryptographic and structural checks pass.

11.2 Detectable failures

Condition	Expected Decoder outcome / interpretation
Exact original bytes + valid metadata/signature/chain	VERIFIED
Video edited, blurred, cropped, re-encoded or replaced	FILE_HASH_MISMATCH / mismatch
Signature or signed metadata changed	INVALID_SIGNATURE
chain_hash or previous_hash modified	BROKEN_CHAIN
Sequence omitted	MISSING_SEQUENCE
Sequence order reversed	REORDERED_SEQUENCE
Duplicate sequence	DUPLICATE_SEQUENCE
Records from the wrong modern capture session	SESSION_MISMATCH

Condition	Expected Decoder outcome / interpretation
No trusted cloud reference	FINGERPRINT_NOT_FOUND / NOT_FOUND

Forensic scope

SHA-256 proves exact byte equality against a trusted reference. It does not identify the semantic type of edit. The report therefore claims detection of modification, not classification of the modification.

12. Security and Access Control

Control	Implementation
Authentication	Supabase Auth
Workspace/device authorization	RLS helper functions and membership/device ownership checks
Fingerprint immutability	Application roles receive SELECT/INSERT; UPDATE/DELETE is revoked
Private incident video	Storage access requires authenticated authorized device/workspace access
Objects overwrite protection	Incident upload uses upsert: false and validates duplicate-object bytes
Device signing identity	ECDSA public key stored with device; private key remains on device
Session isolation	v2 fingerprints require session_id and segment_id with uniqueness constraints

The SQL migration layer is part of the security design. This prevents the integrity model from depending only on frontend behavior. A production version should additionally use hardware-backed mobile keys/attestation rather than browser IndexedDB for the strongest device trust guarantees.

13. Deployment Architecture

13.1 Vercel

The documented production path serves the Vite single-page application through Vercel. The project uses a rewrite configuration so application routes resolve to index.html. Supabase remains the managed backend for Auth, PostgreSQL, Realtime and Storage.

13.2 Docker / Express alternative

The Dockerfile uses Node 22 Alpine, installs dependencies with npm ci, builds the Vite frontend and runs the optional Express server on port 3001. The server exposes /api/health and serves the built frontend when dist is present.

```
docker build --build-arg VITE_SUPABASE_URL=<url> --build-arg VITE_SUPABASE_ANON_KEY=<key> -t clouddash-integrity .
```

```
docker run -p 3001:3001 clouddash-integrity
```

14. Testing and Validation

The repository includes four test files and a CI workflow. Core automated checks cover:

- SHA-256 determinism.
- Valid hash-chain construction and verification.
- Detection of modified hash/metadata and broken previous-hash links.
- Missing, reordered and duplicate sequence behavior.
- ECDSA signature validation and rejection of signature/metadata tampering.
- Session-scoped identity and sequence-zero restart behavior.

- Persistent outbox identity and per-device retry selection.
- Exact video hash comparison.
- Signed manifest validation.
- Retention of locked evidence and deletion of expired normal video.
- Guarded manual deletion.
- Incident storage conflict detection.

Continuous integration configuration:

```
npm ci
npm test -- --run
npm run build
```

15. Runtime Demonstration Evidence

The following screenshots were supplied from the deployed application and are included as runtime evidence. They support the report's description of the implemented user-visible behavior.

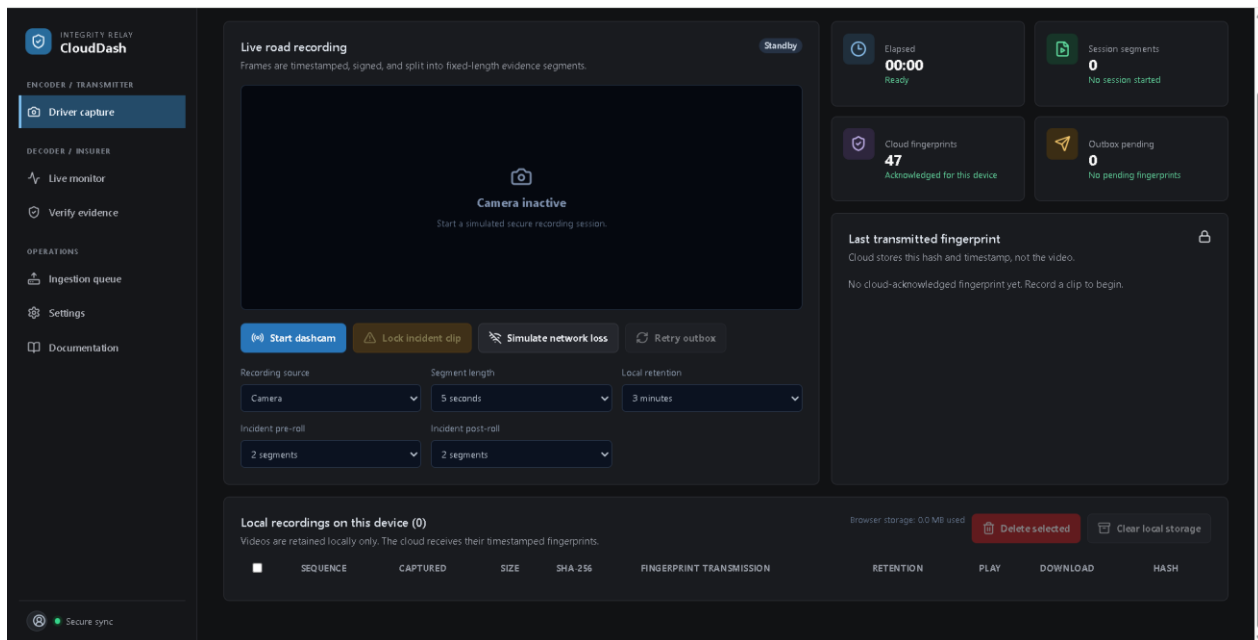


Figure 2. Encoder / Transmitter - Driver Capture interface. The deployed application exposes camera/simulation capture, segment length, retention, incident pre/post-roll, network-loss simulation, retry controls, cloud fingerprint count, and local recordings.

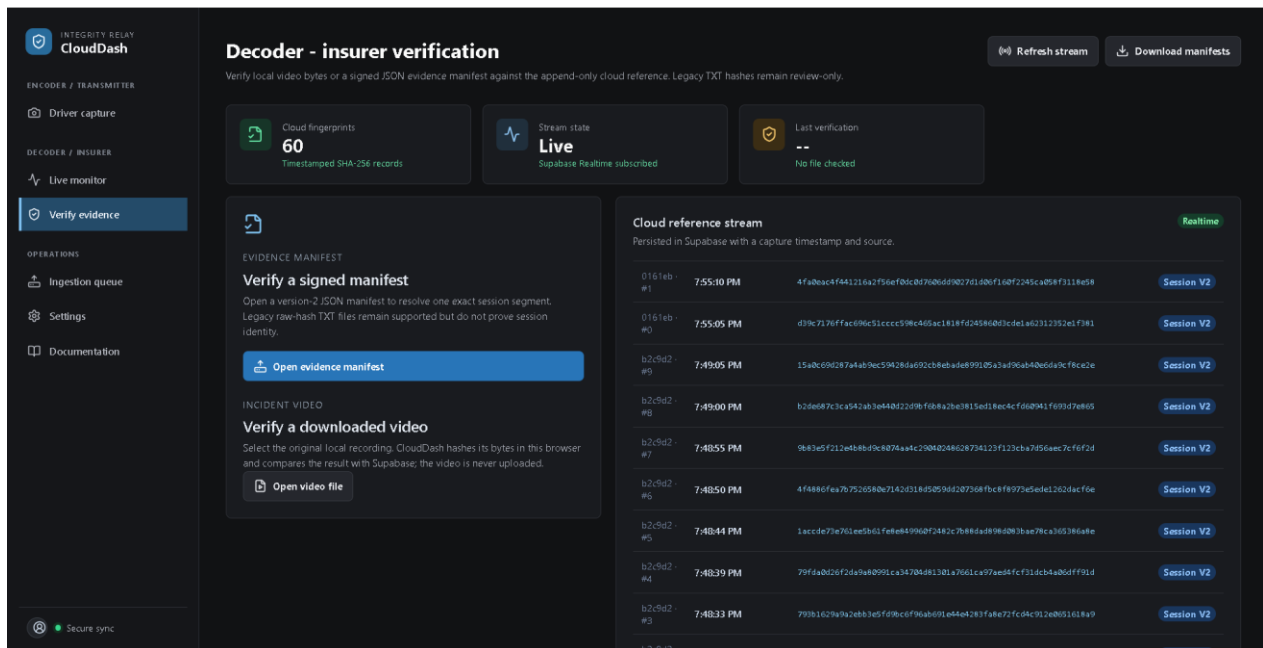


Figure 3. Decoder / Insurer verification interface. The Decoder exposes signed manifest verification, local video verification, the Supabase Realtime reference stream, and session-v2 cloud fingerprints.

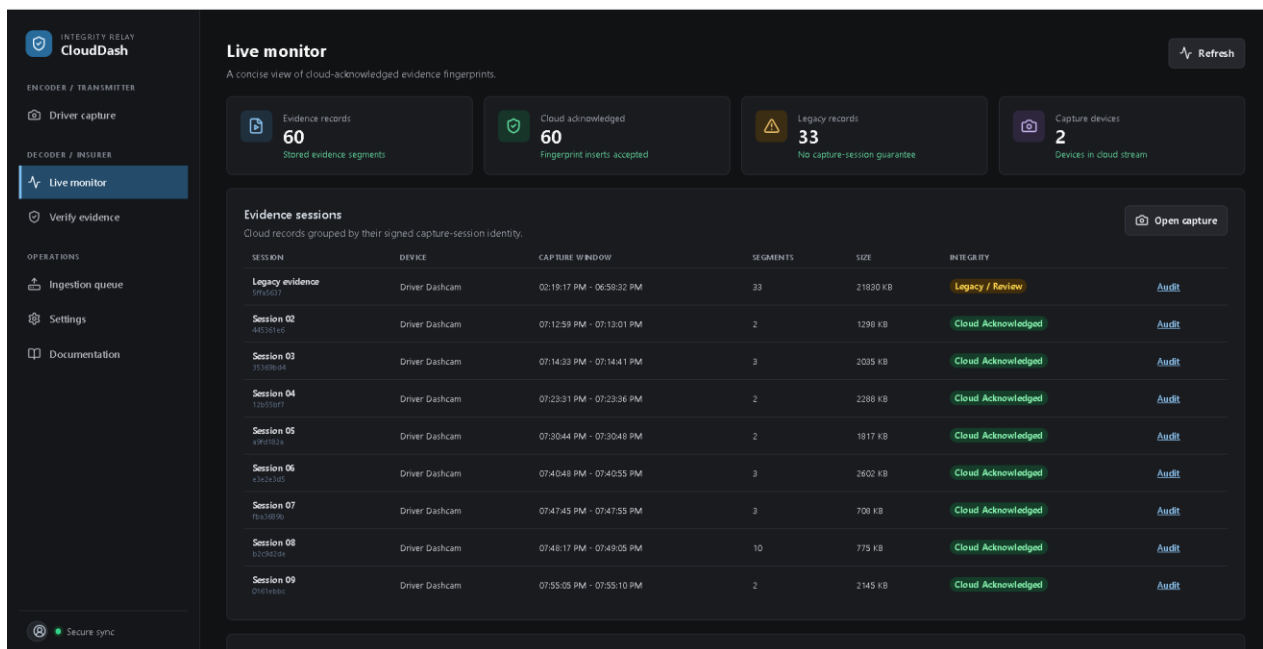


Figure 4. Live Monitor. Trusted cloud records are grouped by signed capture-session identity, distinguishing modern session-v2 evidence from legacy records.

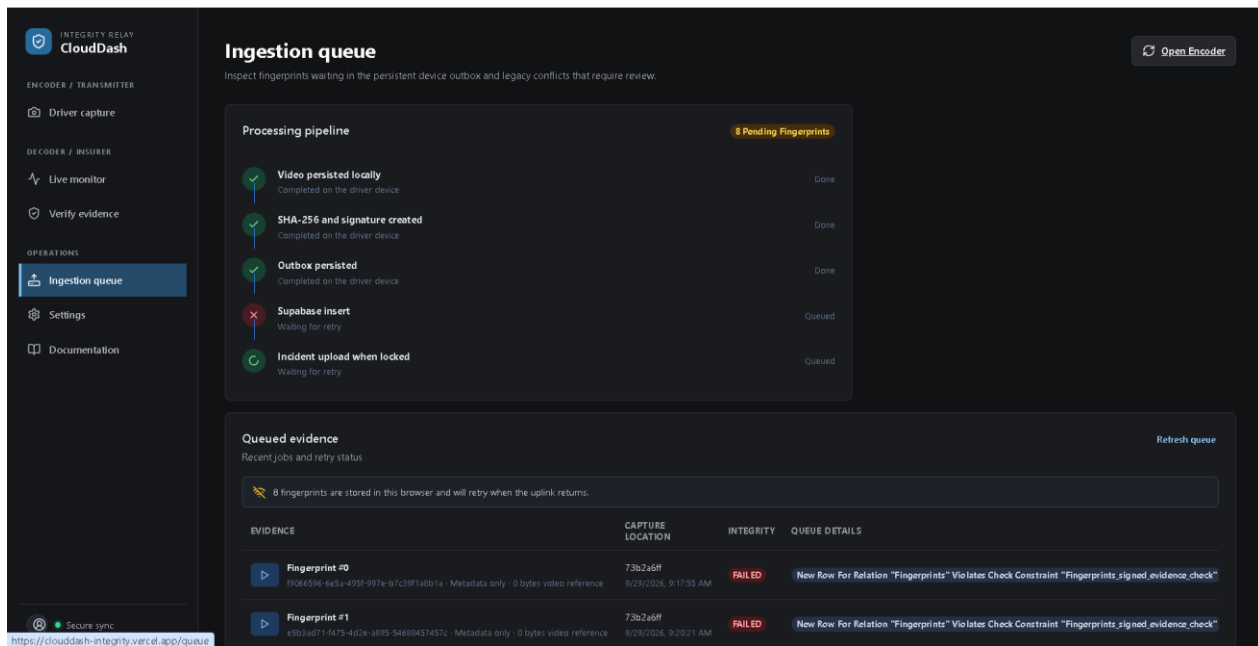


Figure 5. Persistent ingestion queue during a failed upload condition. Fingerprints remain in the browser outbox with failure information instead of silently disappearing.

16. Demonstration Video Procedure

16.1 Encoder / Transmitter recording

1. Sign in and open Encoder / Transmitter -> Driver capture.
2. Select Camera, grant browser permission and record at least two segments on a real computer or smartphone camera. The driving simulation may be shown as an additional source, but it does not replace the physical-camera demonstration.
3. Show the capture-session ID, session-relative sequence, timestamp, byte size and SHA-256 fingerprint for each local segment.
4. Play and download one original local segment. Explain that normal video remains in IndexedDB and only its signed fingerprint is transmitted.
5. Stop and start recording again to show a new session ID, sequence 0 and a null previous hash. Then simulate network loss and allow a segment fingerprint to enter the persistent outbox.
6. Open Ingestion Queue and show the pending/failed item and preserved local state.
7. Restore the uplink and show the preserved fingerprint becoming SENT exactly once, either automatically or through Retry outbox.
8. Refresh the application to show that IndexedDB restores local recordings and queued state.
9. Lock an incident window and show that its video is preserved and made available to the insurer/private storage path.
10. Show retention/manual deletion behavior for non-locked acknowledged evidence.

16.2 Decoder / Insurer recording

1. Open Decoder / Insurer -> Verify evidence.
2. Show the cloud reference stream and session-v2 records.
3. Verify an original downloaded segment and show VERIFIED.
4. Modify a copy by re-encoding, blurring, cropping or changing bytes; verify it and show a file hash mismatch.
5. Use a signed version-2 JSON evidence manifest and explain exact segment resolution, signature validation and session-chain validation.

6. Open public/sample-fingerprints.txt to demonstrate legacy one-SHA-256-per-line parsing. Change one character or add a malformed line and show the review-only failure result. Explain that a raw hash alone does not prove session identity.
7. Retrieve a private locked incident video, play it and verify it. Explain the difference between cloud acknowledgement (SENT) and cryptographic verification (VERIFIED).

16.3 Segment Fingerprinting Rationale

The assignment describes hashes derived from camera frames. CloudDash uses each fixed-duration MediaRecorder blob as the transmitted forensic evidence unit. SHA-256 is computed over the exact container bytes, which include every encoded frame and audio byte in that playable segment. Blur, crop, overlays, frame insertion or deletion, re-encoding, truncation, replacement and other byte changes therefore invalidate the trusted digest. This bounded segment design preserves exact-file integrity while reducing transmission and database overhead compared with one digest per decoded frame. SHA-256 detects that bytes differ; it does not classify the visual edit.

16.4 Legacy TXT Compatibility

For backward compatibility, the Decoder accepts a plain-text file containing one 64-character hexadecimal SHA-256 value per non-empty line. It validates line format and searches the trusted cloud stream for each value. A changed valid-looking hash returns FINGERPRINT_NOT_FOUND or a mismatch; malformed lines return an invalid-file result. TXT input remains review-only because a raw digest contains no signed session, device, sequence or chain identity. Modern evidence exchange uses the signed version-2 JSON manifest.

17. Requirement Traceability

Assignment requirement	Implementation evidence	Status
Acquire frames / camera video	Media capture source and MediaRecorder workflow	Implemented
Compose/process video	Fixed-length segment generation	Implemented
Generate hashes	SHA-256 over exact video bytes	Implemented
Dynamic transmission of hashes	Persistent outbox -> Supabase fingerprints	Implemented
Network interruption handling	IndexedDB queue, retry on reconnect/manual retry	Implemented
Delete data after validity period	Configurable retention; locked evidence exempt	Implemented
Complete Encoder source	React/Vite plus storage/security/repository modules	Implemented
Encoder installation/configuration report	README/docs plus this report	Implemented
Encoder results demo	Documented demo procedure; video submission still required	Submission artifact required
Decoder integrity verification	Hash + ECDSA + session hash chain + trusted reference	Implemented
Algorithm/protocol explanation	Sections 11-12 and repository architecture/security code	Implemented
Detect modified/corrupt/inconsistent evidence	Mismatch, invalid signature, broken/missing/reordered chain states	Implemented
Complete Decoder source	Integrated Decoder/monitor/retrieval modules	Implemented
Decoder installation/configuration report	Same reproducible application/backend environment	Implemented
Decoder results demo	Documented demo procedure; video submission still required	Submission artifact required
Legacy TXT fingerprint handling	One SHA-256 value per line; malformed and changed values fail; raw	Implemented and

Assignment requirement	Implementation evidence	Status
	hashes remain review-only	documented
Third-party attribution	Appendix C lists dependencies, licenses/specifications and official references	Documented
Physical camera validation	Encoder demonstration procedure requires browser permission and real camera capture	Submission demonstration required

18. Known Limitations and Production Hardening

Current prototype boundary	Recommended production improvement
Browser IndexedDB private key storage	Move driver implementation to native Android/iOS and use hardware-backed Keystore/Secure Enclave
Browser/device compromise before capture	Add device attestation, signed enrollment and hardware-backed identity
Exact-byte hashes do not classify edits	Optionally add perceptual analysis as a separate diagnostic layer, without replacing SHA-256 integrity
Operational observability is limited	Add centralized audit logs, alerts, monitoring and verification failure telemetry
Retention policy is configurable in application	Define formal legal/insurer retention and object lifecycle policies
Academic prototype deployment	Perform load, mobile battery, network and browser compatibility testing before production use

19. Conclusion

CloudDash Integrity implements the assignment as a complete hybrid system rather than an isolated hash demonstration. The Encoder captures and segments road video, persists original evidence locally, computes exact-byte SHA-256 fingerprints, binds them into signed session hash chains, transmits them through an outage-tolerant queue and applies retention rules. Locked incidents are escalated into private cloud evidence storage.

The Decoder independently computes the supplied file hash, verifies the device ECDSA signature, validates capture-session sequence/chain continuity and compares the result against the append-only trusted cloud reference. Supabase provides authenticated cloud persistence, RLS, Realtime and private incident storage. The repository also includes reproducible deployment documentation, tests and CI.

For final submission, the remaining non-code deliverables are the narrated Encoder and Decoder demonstration videos showing the behaviors documented in Section 16.

Appendix A. Repository File Map

Path	Purpose
README.md	Project overview, local run instructions and hybrid architecture summary
package.json	Dependencies and development/build/test scripts
src/App.jsx	Primary application and user workflows
src/security/chain.js	Versioned evidence canonicalization, SHA-256 and hash-chain verification
src/security/deviceKeys.js	ECDSA P-256 key generation/signing/verification
src/security/fingerprintFile.js	JSON signed manifest and legacy TXT parsing
src/security/verification.js	File hash and trusted reference verification
src/storage/localEvidenceStore.js	IndexedDB local video and retention
src/storage/offlineQueue.js	Persistent offline outbox and retry
src/fingerprintRepository.js	Supabase fingerprint/session/incident operations

Path	Purpose
src/evidenceRepository.js	Evidence/device repository wrappers
supabase/migrations/	Schema, RLS, storage, realtime and capture-session upgrades
docs/ARCHITECTURE.md	Trust boundaries and evidence data flow
docs/SUPABASE.md	Local/hosted backend setup
docs/DEPLOYMENT.md	Vercel production deployment path
docs/DEMO.md	Encoder and Decoder demo checklist
tests/	Automated integrity and repository tests
.github/workflows/ci.yml	Automated npm install, test and build
Dockerfile	Container deployment option
vercel.json	SPA deployment rewrites

Appendix B. Reproducibility Commands

```
# Install
git clone https://github.com/asrieldev/clouddash-integrity.git
cd clouddash-integrity
npm ci

# Local Supabase
supabase start
supabase db reset

# Hosted Supabase
supabase link --project-ref <project-ref>
supabase db push

# Run
npm run dev

# Validate
npm test
npm run build

# Optional Docker
docker build -t clouddash-integrity .
docker run -p 3001:3001 clouddash-integrity
```

Appendix C. Open Source Attribution and References

CloudDash uses installed open-source packages and standard browser APIs. Exact package versions are locked in package-lock.json. No third-party source code is copied into the application outside those dependencies.

Component	Use	License or standard	Official reference
React and React DOM	User interface	MIT	https://react.dev/
React Router	Client-side routing	MIT	https://reactrouter.com/
TanStack Query	Asynchronous client state	MIT	https://tanstack.com/query/latest
Supabase JavaScript	Auth, PostgreSQL, Realtime and private	MIT	https://supabase.com/docs

Component	Use	License or standard	Official reference
	Storage		
Vite and React plugin	Development and production build	MIT	https://vite.dev/
Lucide React	Interface icons	ISC	https://lucide.dev/
Recharts	Dashboard charts	MIT	https://recharts.org/
Three.js	Driving simulation rendering	MIT	https://threejs.org/
Express, CORS and Concurrently	Optional local and container tooling	MIT	https://www.npmjs.com/
Web Crypto API	SHA-256 and ECDSA P-256	W3C specification	https://www.w3.org/TR/WebCryptoAPI/
IndexedDB, MediaRecorder and getUserMedia	Local persistence and camera capture	Web standards	https://developer.mozilla.org/docs/Web/API
Node.js test runner	Automated evidence tests	MIT	https://nodejs.org/api/test.html

Appendix D. Final Submission Checklist

- Final project report in DOCX and PDF formats.
- Complete source code and GitHub repository URL.
- Deployed Vercel application URL.
- Narrated Encoder / Transmitter demonstration video.
- Narrated Decoder / Insurer demonstration video.
- Physical camera or smartphone capture shown in the Encoder demonstration.
- Test-account instructions shared separately without committing credentials.
- Successful npm test and npm run build results.